

F24-080-D-LeafSpec

Project Team

Abdul Rafay Sheikh	21I-2582
Muhammad Osama Fahim	21I-0439
Ahmad Rafiq	i21-0439

Session 2021-2025

Supervised by

Dr. Labiba Fahad



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope.....	1
1.3	Modules.....	1
1.3.1	Module 1.....	2
1.3.2	Module 2.....	2
1.4	User Classes and Characteristics	2
2	Project Requirements	5
2.1	Use-case/Event Response Table/Storyboarding	5
2.2	Functional Requirements.....	5
2.2.1	Module 1.....	5
2.2.2	Module 2.....	6
2.3	Non-Functional Requirements.....	6
2.3.1	Reliability	6
2.3.2	Usability.....	6
2.3.3	Performance.....	6
2.3.4	Security	7
2.4	Domain Model	7
3	System Overview	9
3.1	Architectural Design	9
3.2	Design Models	9
3.3	Data Design.....	10
	References	11

List of Figures

1. Use Case Diagrams:

- 1.1:** Upload Leaf Image
- 1.2:** Preprocess image.
- 1.3:** Classify species using CNN.
- 1.4:** Display species information.
- 1.5:** View previous classification.

2. Domain Model

3. Architectural Diagram

4. Design Models

5. System sequence diagram

6. State Transition Diagram

7. Entity Relationship Diagram

List of Tables:

- **User Classes and Characteristics**

Chapter 1

Introduction

This document is on a leaf species detection application product. The app uses CNN (custom build) to recognize the species by the leaf photo. It is for the plant enthusiasts, botanists, researchers those who can help in reducing the tiring and tedious work of manual leaf classification.

1.1 Problem Statement

Identifying a plant without any automated system is tiring and tedious task as they have a similar structure and features which are not visible to naked eye, it can lead to errors and incorrect identification which can result in inconsistencies for people who use this information for critical tasks such as creating medicines which is a very critical task and there must be no margin for error as it could become a matter of life and death. Also Botanists can also speed up their work which can become one click away and make them research and study work must more effective and efficient.

1.2 Scope

- The project will focus on developing a mobile app to accurately classify plant species using leaf images.
- The application aims to be better than existing competitors in terms of accuracy, intuitiveness and ease of use.
- The application would provide useful information about the plant species such as suitable region, basic information and short history to help both enthusiasts and botanists.
- The application would be developed using Flutter to facilitate convenient multi-platform development and deployment.

1.3 Modules

1.3.1 Module 1

Training the model: This Module focuses on Pre-Processing the images and extracting unique features for creating a working model which would be able to distinguish plant species based on leaf images.

1. Data Preprocessing
2. Training Model on dataset of approximately 10-15 species

1.3.2 Module 2

App Development: This Module focuses on creating an application which streamlines the integration of a working model with a multi-platform application.

1. Integrating Model with Application
2. Creating Admin dashboard
3. Creating User dashboard

1.3.3 Module 3

Testing and improvement: This module focuses on testing the model thoroughly and improving model's accuracy by catering more species and tuning hyper parameters.

1. Increasing the dataset and catering more species.
2. Testing the features

1.4 User Classes and Characteristics

User class	Description
End User	End users are botanist, researcher or person interested in environmental things who use the application to recognize the plant species based on leaf Images. They Upload images from multiple devices (smartphones, tablets or computers) and feedback will be expected upon the classification results. They are interested in using a product that is user friendly and accurate.
Developer	The app and its various parts are what the developers create, you own it so now it needs to be developed and maintained. They will deploy deep learning models, design the user interface and maintain working of the system. The developers will require the source code and documentation for building features and debugging them next. There will be several updates to make it more accurate and better over time.
Testers	Testers ensure that the application is working properly and complies with all specifications. They will conduct unit tests, integration tests, user acceptance tests etc to ensure that everything works as expected. They concentrate to find bugs and assure a quality UX. It will test the whole application flow like models, UI this will cause seamless process.

Chapter 2

Project Requirements

2.1 Use-cases

1. Upload leaf Image:

- **Actor:** User
- **Description:** User uploads the image of a leaf to the system.
- **Precondition:** The user has made an account and is logged into the application.
- **Main Flow:**
 1. User selects a leaf image from gallery
 2. User uploads the leaf image for classification
 3. The system processes the image
- **Postcondition:** The image is processed and classifies the leaf image into a specie.
- **Alternate Flow:** If incorrect image is uploaded and accuracy is very low then error message shall be displayed to the user.

2. Preprocess Image:

- **Actor:** System
- **Description:** Image is resized and augmented by the system to make it classification ready.
- **Precondition:** Image has been correctly uploaded and preprocessed for the classifier.
- **Main Flow:**
 1. Image is resized for the CNN.
 2. Image may be augmented to increase the dataset.
 3. The image is passed to the CNN so that specie classification can be done.
- **Postcondition:** Image is sent to the classifier.
- **Alternate Flow:** Error or exception is thrown when preprocessing cannot be done.

3. Classify Species using CNN:

- **Actor:** System
- **Description:** Custom Trained CNN is used by the system to classify species according to the provided leaf images.
- **Precondition:** Correctly preprocessed image has been received.
- **Main Flow:**

1. CNN receives the preprocessed image
 2. CNN classifies the specie and gives a confidence score indicating how certain it is.
 3. Classified Specie with a confidence score is returned to the user.
- **Postcondition:** Specie identification along with confidence score is done.
 - **Alternate Flow:** Error message is displayed if confidence score is below (20 %).

4. Display Species Information:

- **Actor:** User
- **Description:** The system also displays the specie related information to the user
- **Precondition:** Specie has been identified and confidence score has been generated.
- **Main Flow:**
 1. System fetches specie data from the database.
 2. Specie name, confidence score, basic information regarding the specie is shown to the user.
- **Postcondition:** User views the identified specie along with the description.
- **Alternate Flow:** If confidence score is below 20 percent an error message is displayed and consequently no information or identified specie is displayed.

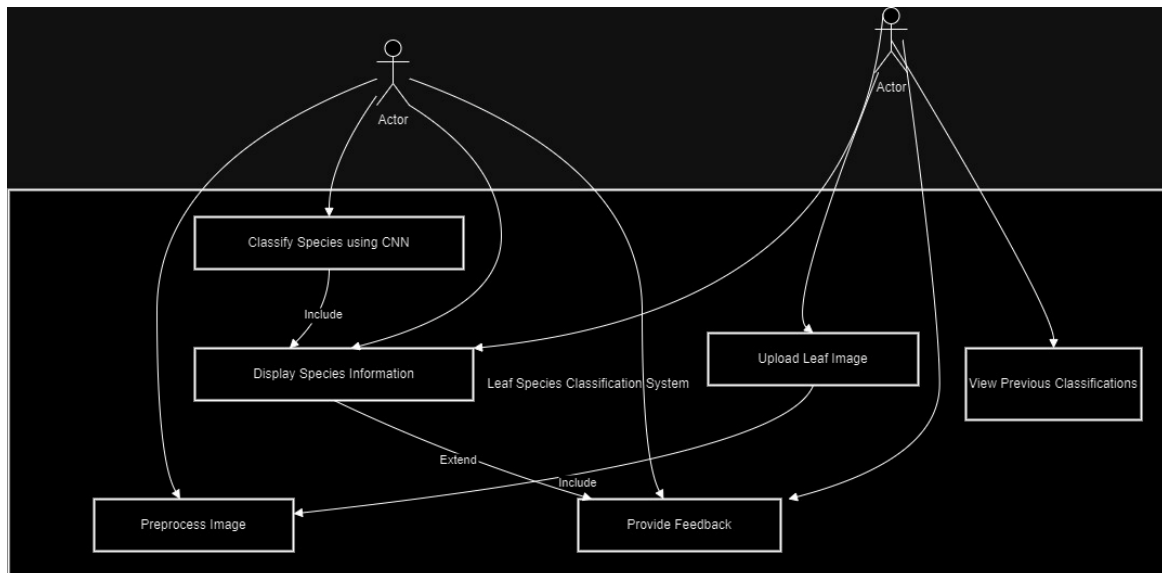
5. Provide Feedback:

- **Actor:** User
- **Description:** Feedback is provided by the user after his provided leaf image is classified by the system.
- **Precondition:** The identified specie name and information has been reviewed by the user.
- **Main Flow:**
 1. User provides feedback regarding the identification of the specie and can also and can also provide comments on how improvements can be made.
 2. The feedback is stored in the database so that developers can review them later.
- **Postcondition:** Feedback is stored in the database and “Thank you for the feedback (or some similar message) is shown to the user”.
- **Alternate Flow:** An exception is thrown if feedback is unable to be stored in the database.

6. View Previous Classifications:

- **Actor:** User
- **Description:** User can see the classifications done in the past.
- **Precondition:** User has used the application for classifications before.
- **Main Flow:**
 1. User can select an option for previous classifications.
 2. User views previous classifications, confidence scores and relevant regarding to the species classified in the past.
- **Postcondition:** User vies previous classifications.
- **Alternate Flow:** User is shown empty history as no classifications were done by the in the past.

Use-case Diagram:



2. Project Requirements

2.2 Functional Requirements

2.2.1 Module 1

Following are the requirements for module 1:

Data Preprocessing:

- **FR1:** Leaf image provided by the user shall be resized to a standard resolution which is accepted by the CNN.
- **FR2:** Data cleaning shall be done so that model's accuracy can be improved

Model Training:

- **FR2:** The system shall train the model on Preprocessed dataset of 10-15 species approximately.
- **FR3:** Model's testing and validation accuracy along with confidence scores shall be used to judge the model's overall accuracy.
- **FR4:** Updated Model shall be saved to the database.

2.2.2 Module 2

Following are the requirements for module 2:

User Interface (UI) development:

- **FR5:** The user interface shall be friendly, easier to use and have intuitive navigation. Learning curve should be minimal.
- **FR6:** Output shall contain identified specie along with confidence score and description.

Admin and User Dashboards:

- **FR9:** Admin Dashboard shall be used to allow admins to add, remove or delete species or perform maintenance of the application.
- **FR10:** The application shall have a user interface for identification of species as well.

Integration of Model with Application:

- **FR11:** The application shall be integrated with the CNN model so that leaf images could be send to it for classification.

2.2.3 Module 3

Following are the requirements for module 3:

Testing the Model:

- **FR12:** Model shall be regularly evaluated using a testing dataset.
- **FR13:** Error message shall be provided by the system if the confidence score is less than 20 percent indicating that image sent to it is unclear.

Improvement of Model:

- **FR14:** The model's accuracy should be improved by catering more species into the dataset as well as enhancing the quality and quantity of the dataset.
- **FR15:** The system shall use Feedback to improve the application as users would explain the issues that they might be facing.

2.3 Non-Functional Requirements:

2.3.1 Reliability

Reliability of the system will be prioritized and will be gauged by an MTBF (Mean Time Between Failures), with a target of 500+ operation hours in optimal and normal conditions

- **Definition of Failure:**

Whenever uploading images failed, leaf classification failed, or a user did not get the result after 2 minutes maximum.

- **Error Detection Strategy:**

If the classification accuracy of the system falls below a certain value (e.g., 20%), an error message will be shown to the user, and he is given suggestion on what to do next.

- **Error Correction Strategy:**

If an error is detected by the user, re-processing of the image will be done and if the failure is still here there, then an error message is displayed, and user is also guided on what needs to be done next (probably prompting to upload a better image and try again)

- **Failure Consequences:**

If the failure occurs, leaf image would not be classified, Data integrity would still be maintained and images would be stored for later inspection.

- **Failure Recovery:**

If a failure happens and conditions are critical then system shall automatically restart and give an indication to the development team and tell the user when to be back as maintenance would be going on.

2.3.2 Usability

Usability requirements make sure that user interactive interface which is easy to learn and navigate. Below are some functional requirements which tend to enhance user experience.

- **Ease of Use:**

Use-1: Interface must be intuitive and shall require minimum number of clicks (approximately 3 - 5) to upload leaf images.

Use-2: Application buttons must be clearly labeled to avoid user confusion easy navigation.

- **Minimal number of pop-ups (Taking example of Feedback Mechanism):**

Use-3: The feedback mechanism shall not appear as a pop-up every time user gets classification results, instead there shall be an option of “Give Feedback” for un-interrupted experience of the user.

- **Accessibility:**

User-4: The application shall be accessible to users using different platforms (IOS, Android, Web) so that device compatibility is ensured.

2.3.3 Performance

General Performance goals:

Responsive user experience along with maintenance of accurately identifying species data handling.

Specific Performance goals:

- **Species Classification using CNN:**

PER-1: Custom trained CNN shall be able to classify species within few seconds after getting input from the user.

- **Data Retrieval from Database:**

PER-2: Data Retrieval shall be done within (5 – 10 seconds) from the database so that quick and timely response is provided to the user

- **User Feedback Submission:**

PER-3: The system shall store feedback submission right after users clicks on “Send Feedback” button and give a message “Thank you for the feedback” after storing the data.

- **Previous Classification history:**

PER-4: The system shall be fast enough to retrieve data from the Database within 5 – 10 seconds and display to the user as quickly as possible.

- **Error and Exception Handling:**

PER-5: Error messages shall be displayed for issues such as unclear image given by the user, too much waiting time (approximately 2 minutes) in user interaction, unable to pre-process image as soon as issue is identified by the user.

2.3.4 Security

The system shall encrypt all sensitive user data, such as passwords, email, user history (previous classifications), In this one the data would get protected in case of unauthorized access.

- **Confidentiality:**

As the data would be stored after encryption, confidentiality of the users would be maintained.

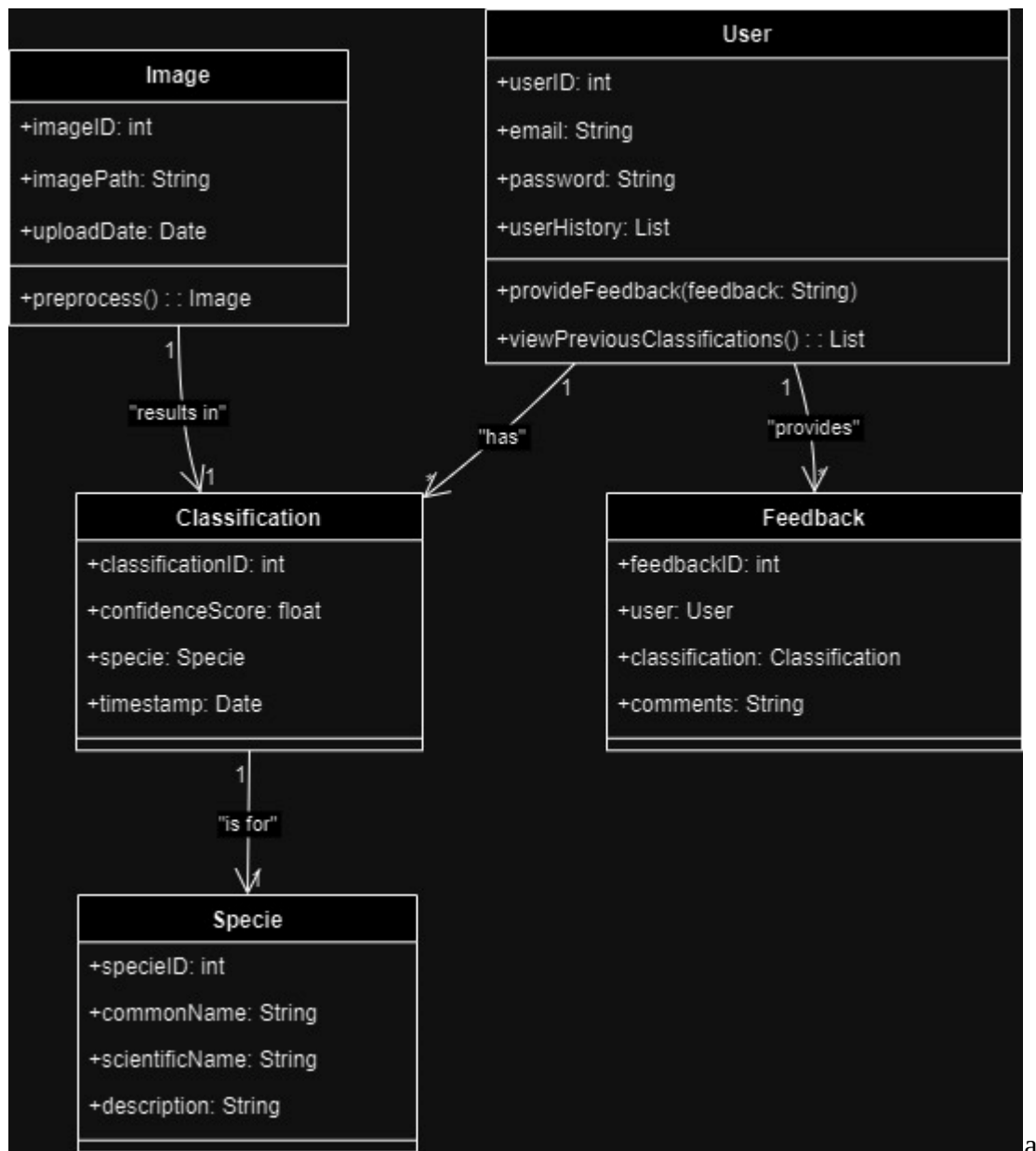
- **Detecting of malicious Brute Force attempt:**

The software would be locked for the user for approximately 10 – 15 minutes, if a user unsuccessfully tries to login to the system more than 10 number of times indicating a chance of unauthorized access.

- **Captcha System:**

Captcha system shall be implemented, to avoid bots to violate the system. This would avoid any DDOS attack attempts on the system.

2.4 Domain Model



Chapter 3 System

Overview

System Design:

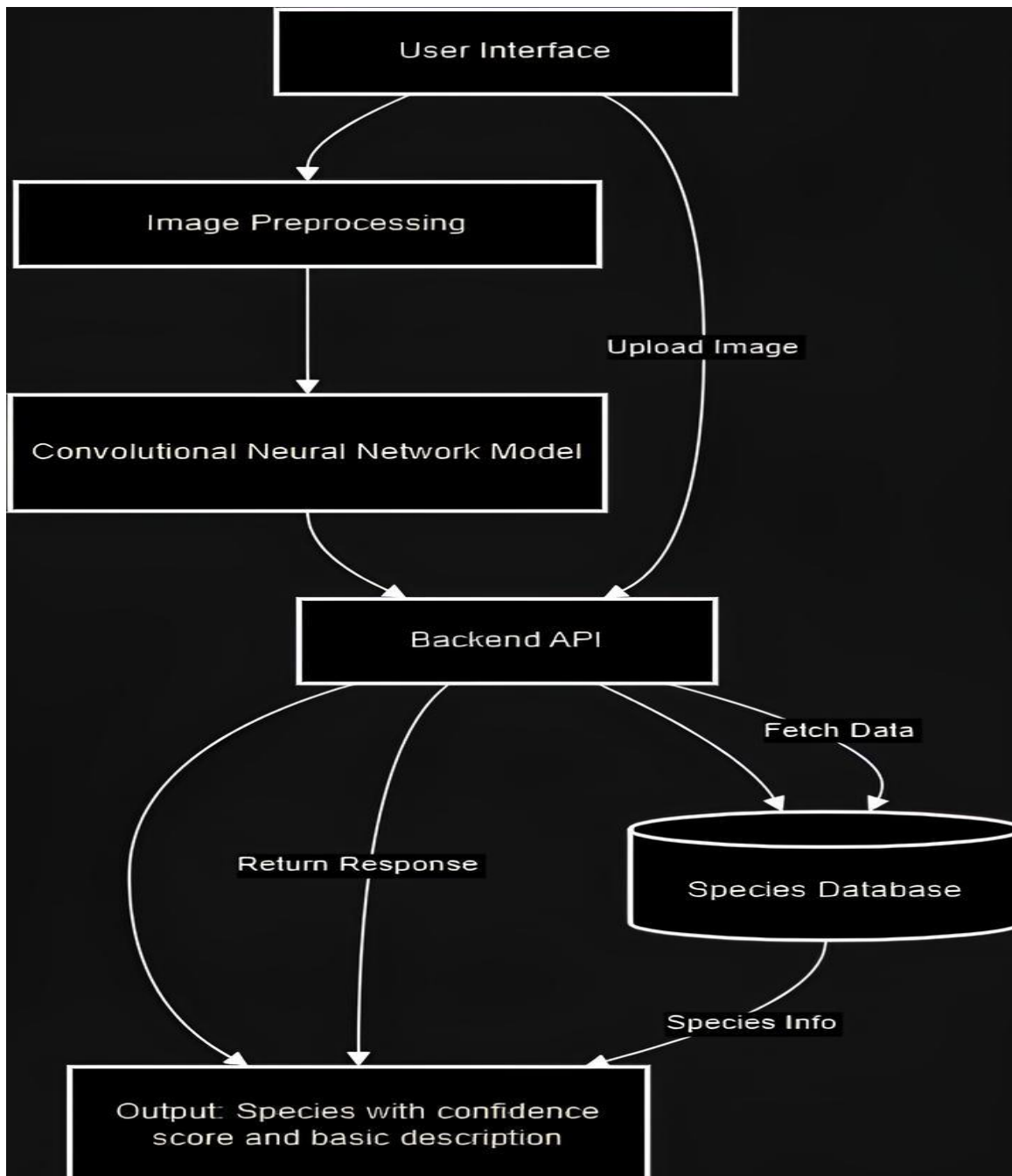
We will be using Object Oriented Programming Design for our project

Justification:

We will be adopting an object-oriented approach since it allows us to achieve modularity allowing us to isolate each functionality of the app within its own class and allows us to easily model and implement real world entities such as images, users and databases. Furthermore, OOP also promotes reusability allowing us to reuse the same across different classes.

3.1 Architectural Design

- Architectural Diagram



- **Description:**

As shown in the diagrammatic representation, the **microservices architecture** of a system designed to identify plant species from leaf images returning basic descriptions has components very well modularized, thus supporting individual development, scaling, and maintenance.

- **User Interface:**

Description: This Component represents the Front-end application where users can upload images of

the leaves.

Function: User Interface is responsible for streamlining the entire process of collecting the image, sending it to the classifier and displaying output to the user.

- **Image Preprocessing (IP):**

Description: This microservice processes the image received from the user.

Function: Image Preprocessing resizes, normalizes image to be ready for the classification by the model.

- **Convolutional Neural Network (CNN) model:**

Description: This microservice implements a deep learning model for Species classification.

Function: CNN outputs the resultant specie along with the confidence score which tells how sure the model is regarding a specie.

- **Backend API:**

Description: This microservice provides communication between user interface, CNN model and database.

Function: Backend API takes the image from UI and gives it to CNN, fetches data from the database and returns the response to the UI after prediction.

- **Species Database:**

Description: This microservice stores the description which will be provided along with the classification.

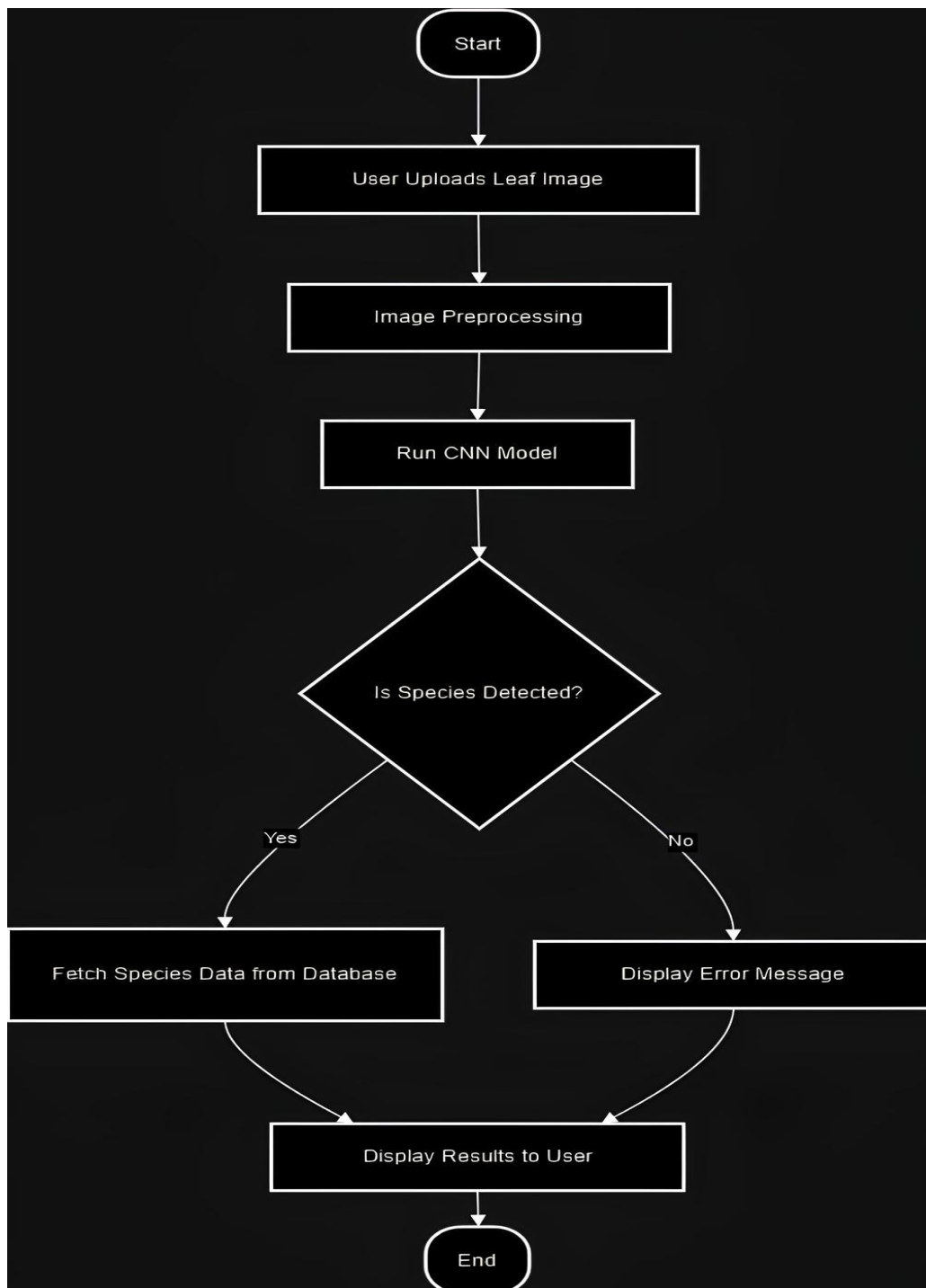
Function: Database stores the description, images and names, it is queried by backend API to retrieve description of the classified specie.

3.2 Design Models

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

Design Models for Object Oriented Development Approach:

- **Process Flow (Activity Diagram)**



Description

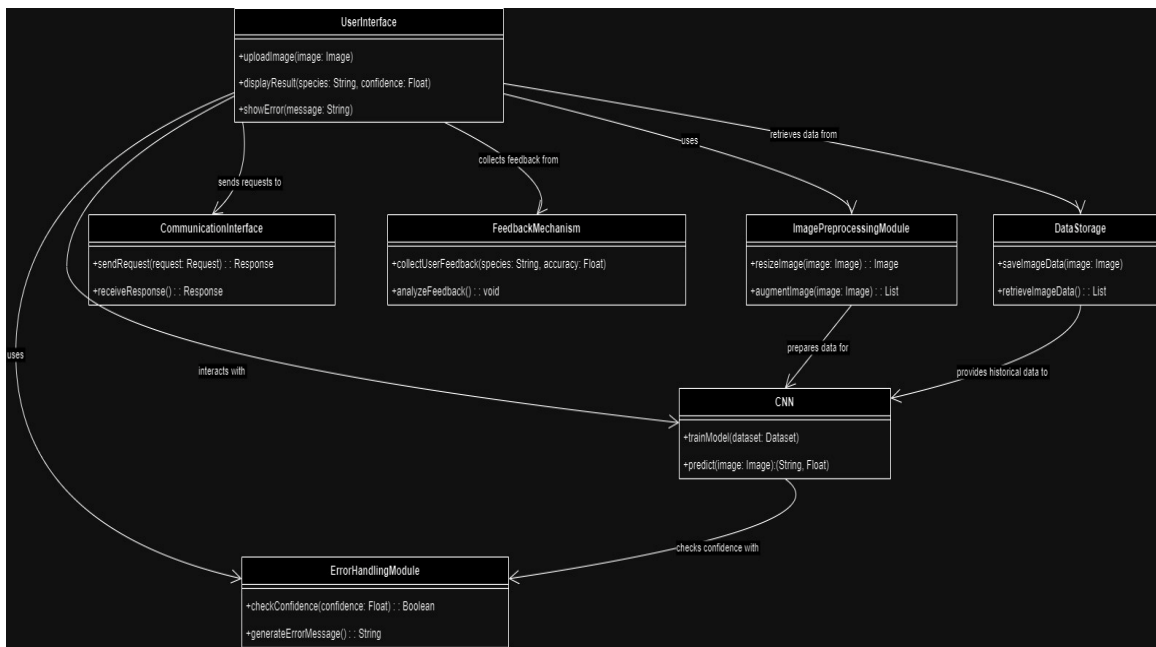
- **Start:** Activity starts when user uploads image of a leaf.
- **Uploading Image:** Interaction of the system and the user is done when users send leaf image for classification via UI.
- **Image Preprocessing:** After image is send by the user it is resized so that the image is correctly accepted by the CNN.
- **CNN Model Running:** The preprocessed image passes through the CNN for identification.
- **Decision Node:**
 - Yes:** If correct identification of the specie is done, the fetch data from the Database to display to the

user.

No: If image quality is poor or the specie is not detected, an error is shown “Please upload a clearer image”.

- **Retrieve Data:** After successful detection of the specie, relevant information is returned from the database.
- **Display:** After successfully detecting the specie and fetching data from the database display the name of the specie and required information to the user.
- **End:** Activity ends when required information is shown otherwise an error message is shown to the user.

- **Class Diagram:**



Description:

The Class Diagram gives the structure of the system and divides the system into smaller modular parts.

User Interface:

- **uploadImage(Image):** This function take image from the user and send it to the CNN for classification.
- **displayResults() -> return (Specie name, score):** This function gives the name of the classified specie along with the confidence score indicating how much confident the model is in predicting the result.
- **showError(message):** if the confidence score is less than 20 percent or due to any other critical issue the message is sent to the function as parameter so that it can be displayed to the UI.

Communication Interface:

- **sendRequest(request: Request):** Sends request from user side to other components of the system such as Database and CNN.
- **receiveResponse():** Receives the response and performs the function which was requested.

Feedback:

- **collectFeedback (Specie, score, User):** This function collects feedback from the user regarding the specie which was classified by the user.
- **analyzeFeedback():** Function analyzes the user, specie which was classified and stores the data into the database accordingly.

Error Handling:

- **checkConfidence(confidence: Float):** -> Boolean: check the confidence score with the threshold.
- **generateErrorMessage():** if the confidence score is less than 20 percent throw an error.

Image Processing:

- **resizeImage(image: Image):** -> **return Image:** Resizes the image to a resolution which is accepted by the CNN.
- **augmentImage(image: Image):** -> **vector<Image>:** Augments the image if specie has a smaller number of images compared to other species and return a list of augmented images.

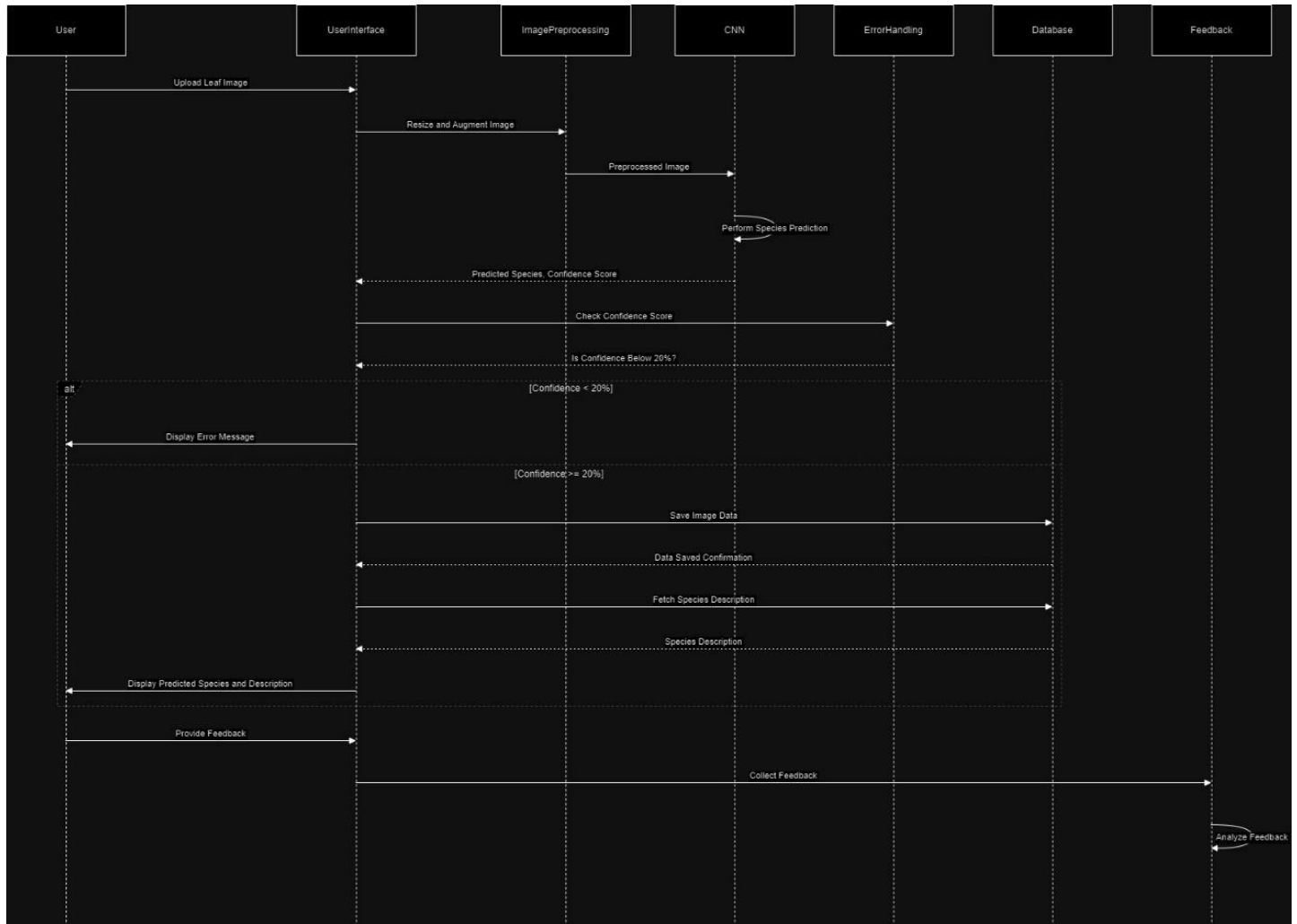
Database (Data Storage):

- **saveImageData(image: Image, user: User):** This function saves the image which was used by the user for classification along with User information in the database.
- **retrieveImageData():** -> **return -> vector<ImageData>:** This function retrieves the images stored in the database for further use for example it can be shown to the user for history of previously classified images.

Convolutional Neural Network (CNN):

- **trainModel(dataset: Dataset):** trains the custom CNN using tensorflow library, includes the link of the dataset
- **predictImage(image: String):** -> **returns (string, double):** This function passes the image to CNN and returns the predicted specie name along with confidence score.

- Sequence Diagram:



Description

Lifelines:

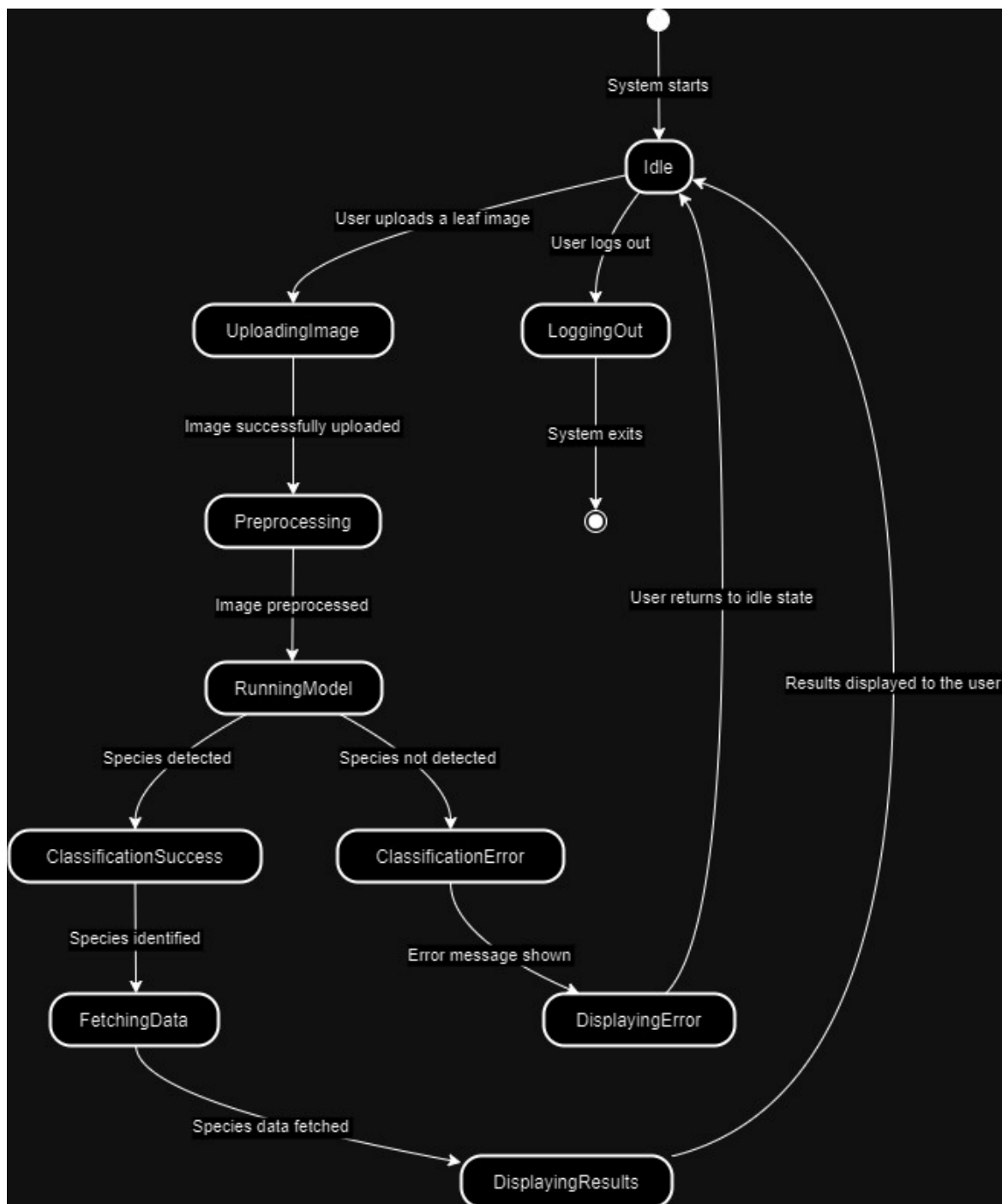
- **User:** Starts the process by uploading a leaf image.
- **User Interface (UI):** UI provides interaction between the system and the user.
- **Image Processing:** Resizes images as well as augments them if necessary.
- **CNN:** Custom trained Neural network classifies the specie.
- **Error Handling:** Gives error to the user if unclear image is provided by the user or confidence score is less than 20 percent.
- **Database:** Database is used to store information regarding a specie and retrieve it when required to display it to the user.

- **Feedback:** Stores update provided by the user after classification of a specie.

Sequence Flow:

- User uploads a leaf image.
- Image is resized so that it becomes ready to be sent to the CNN for classification.
- CNN identifies the specie using leaf image provided to it by the user.
- If confidence score is less than 20 percent, it indicates that unclear image was sent by the user eventually giving an error.
- Feedback provided by the user is stored in the database to use them for future improvements.

State Transition Diagram



Description:

State Transition diagram is used to display the flow of events that occurs in the lifespan of a software, basically explaining transition between states while interaction of the user with the system and explains how backend processes are handled such as sending the leaf image to the classifier, classification, preprocessing etc.

Explanation of components:

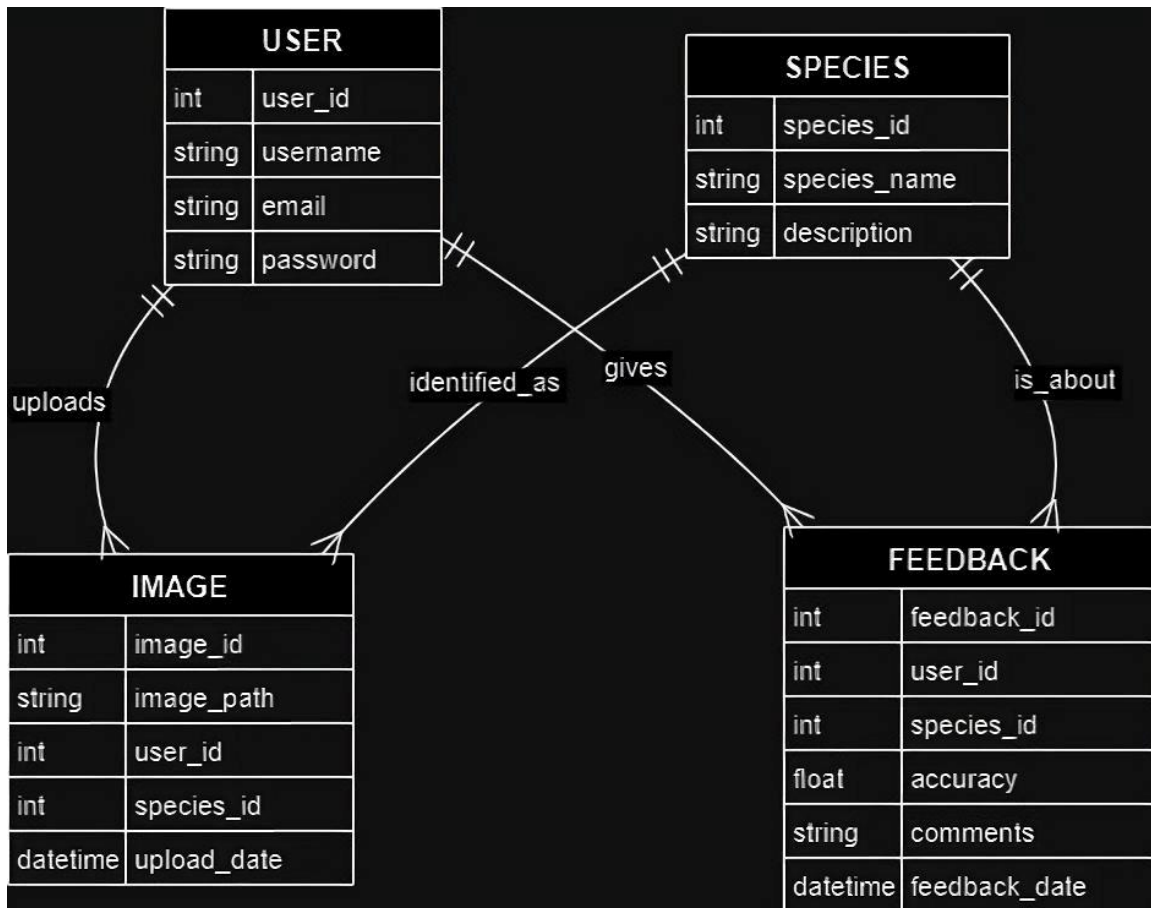
- **Idle:** In this state the system remains in an idle state until the user sends the leaf image to the

system.

- **Upload Image:** When the user sends the image for classification, the system goes transitions to uploading state where it takes an image sent to it by the user.
- **Preprocessing:** Image then preprocesses (resizes) so that it can be sent to the classifier thus system transitions to preprocessing state.
- **CNN Model:** The system now transitions to prediction state where CNN runs to identify the specie on a preprocessed image.
- **Classification:** If the confidence score is less than 20 percent it transitions to error handling state else it transitions to data fetching state.
- **Fetching Data:** After successful classification, the system transitions to Fetching state where it fetches the species information from the database.
- **Display:** After the data is fetched the system goes to Display state where it displays the specie name along with relevant information.
- **Feedback state:** If the user decides to give feedback the program may go to feedback state and then to storing data state and this thing remains consistent with other features as well.

3.3 Data Design

Entity Relationship Diagram (ERD)



Description:

Description includes describing the entities designed and their relations with each other

Entities:

User: This entity represents the users using the application containing information such as:

- User_id (integer)
- Username (string)
- Email (string)
- Password (string)

Image: This entity represents the image uploaded by the user to be classified containing information such as:

- Image_id (integer)
- Image_path (string)
- User_id (integer)

- Species_id (integer)
- Upload time (datetime)

Species: This entity contains the species which will be classified by CNN and are in the used dataset containing information such as:

- Species_id (integer)
- Species_name (string)
- Description (string)

Feedback: This entity stores the feedback provided by the specific user containing information such as:

- Feedback_id (integer)
- User_id (integer)
- Species_id (integer)
- Accuracy (double)
- Comments (string)
- Feedback_date (datetime)

Relations:

- A **User** uploads many **Images** (one to many)
- An **Image** is classified as a **Specie** (one to one)
- A **User** gives **feedback** on the classification of species (one to many)
- **Feedback** is related to one **Specie** classification (one to one)

Bibliography

Newly used dataset links:

- <https://www.kaggle.com/datasets/csafrt2/plant-leaves-for-image-classification>
- <https://www.kaggle.com/datasets/marquis03/plants-classification>
- <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>